

Enhancing Malware Detection through Intent Subsequence Mining from API Call Sequences

Tianheng Qu^{1,2}, Xiurong Cao³, Hao Dai^{1,2}, Hongsong Zhu^{1,2}, Jiayu Zhang^{1,2},
and Kaixiang Liu^{1,2*}

¹ Institute of Information Engineering, Chinese Academy of Sciences, Beijing, China
{qutianheng,daihao,zhuhongsong,zhangjiayu,liukaixiang}@iie.ac.cn

² School of Cyber Security, University of Chinese Academy of Sciences, Beijing, China

³ Beijing Institute of Specialized Machinery, China
caoxiurong0200@163.com

Abstract. Dynamic malware analysis is a key technique for improving software and system security. However, current research often focuses on extracting temporal relationships between APIs or parameter values such as strings and integers, while neglecting virtual address values present in the parameters. This oversight leads to the loss of potential semantic information in detection models. In this paper, we propose an intent-subsequence method for processing Windows API parameters, using otherwise unstable virtual-address values to construct resource-associated contextual relations among API calls. By integrating the raw API call streams with intent subsequences, we construct a multi-dimensional semantic heterogeneous graph and employ a Heterogeneous Graph Transformer for feature learning. Evaluations on three independent datasets demonstrate that our method improves performance across malware-related classification tasks.

Keywords: Malware analysis, API call sequence, API parameters, Graph neural networks.

1 Introduction

As computer viruses, malware, and other cyber threats continue to evolve, their attacks have become increasingly sophisticated, stealthy, and diversified, posing escalating challenges to traditional detection methods when confronting unknown samples and variants. Most recent research has used malware behavior detection models to improve detection accuracy and the ability to recognize unknown malicious activities [19]. This method involves using deep learning techniques to model the behavior of malware [50]. It requires extracting effective feature representations from APIs and converting them into feature vectors that the model can learn from. This process, referred to as feature engineering, is crucial for determining model performance. Missing behaviors during dynamic feature extraction can significantly impact detection performance [26].

* Corresponding author.

Currently, in applying feature engineering to APIs, most methods focus on extracting API names and their temporal relationships as features. However, the temporal relationships represented by the APIs in sandbox reports are not entirely accurate. Even when the same program is executed twice in the same environment, the system call traces can differ significantly [32]. Additionally, function calls in API patterns do not always appear consecutively within a client method, and other function calls may be interspersed [28]. Therefore, if the detection model relies on temporal features that are not accurate, it will inevitably have a negative impact on the model’s accuracy.

Moreover, relying solely on API calls may not adequately reflect the intent of function execution. Neglecting API parameters can result in the loss of critical security-relevant semantic information [24, 35, 45]. In response, some researchers have encoded API parameters, such as strings and integers, into features for the model to learn, which significantly improves the model’s performance [17, 31, 49]. However, the prevalence of virtual address values in API parameters is often overlooked. This is because virtual address values are constrained by the operating system allocation and dynamically change across different execution environments or cycles. Directly inputting these raw virtual address values into the model only increases feature space dimensionality or leads to overfitting, without adding meaningful semantic information.

To address these challenges, we propose a simple yet effective method that uses virtual address values in API parameters to extract intent subsequences from the API call sequences. This method uses shared parameter values to construct resource-associated contextual relations among API calls. Each extracted subsequence represents a resource-associated behavioral pattern among API calls that share the same logical resource. Here, “intent” is used in this operational sense and does not imply direct recovery of a programmer’s high-level semantic intent.

After processing the intent subsequences, we map the API call sequence into a graph representation and propose a heterogeneous graph learning model. Specifically, we employ a Heterogeneous Graph Transformer (HGT) [30] to model the multi-relational semantics and heterogeneous structural dependencies in the API call graph, enabling the model to more effectively capture the behavioral patterns and resource-associated relations induced by different resource-use workflows.

Our main contributions are as follows:

- We propose an intent-subsequence extraction method and apply it to process Windows API parameters, using otherwise unstable virtual-address values to construct resource-associated contextual relations among API calls.
- We map API call sequences into heterogeneous directed graph representations and design an intent-aware heterogeneous graph learning model. Based on this representation, the HGT is used to learn semantic interaction patterns under different edge categories, thereby preserving the structural information and resource-associated behavioral relations embedded in API call sequences and enhancing the model’s ability to identify critical malicious patterns.

- We evaluate the proposed method using three datasets: two publicly available benchmarks and a self-built dataset containing both malware and benign samples, which accounts for the varying quality of reports across different sandboxes. To assess the effectiveness of our approach, we conduct both representation-level analysis and downstream malware classification experiments.

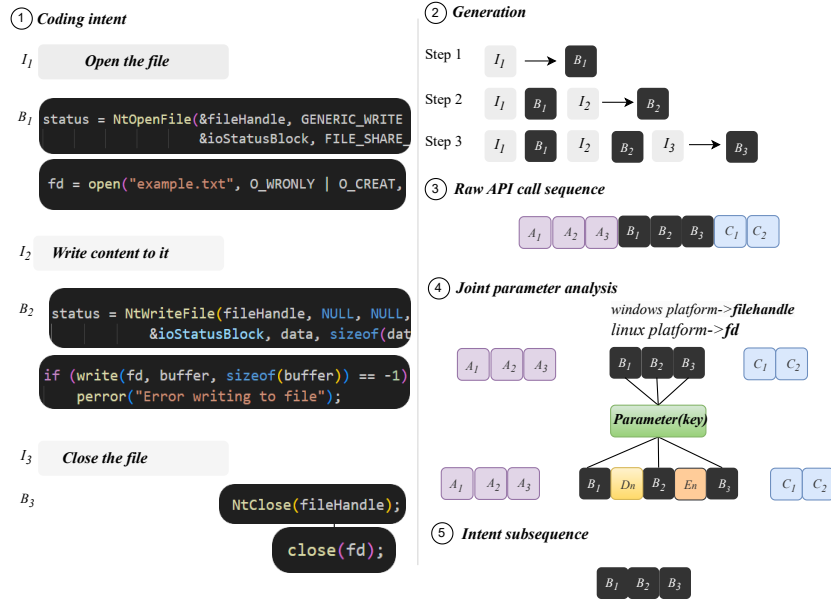


Fig. 1: The key idea of the intent subsequences method.

2 Related Work

Early API-based malware detection methods treat execution traces as sequences and rely on handcrafted features. Zhang et al. [53] encoded API names, categories, and parameter values such as file paths and registry keys using hashing, while Agrawal et al. [13] represented API names and parameter strings with frequent n-grams. These methods are simple and efficient, and they can exploit raw parameter content to a certain extent. However, they depend heavily on manual feature design and mainly model local sequential patterns, which makes it difficult to preserve structural dependencies among API calls.

To better capture execution structure, several studies introduced graph-based representations. Li et al. [35] constructed API call graphs and applied a graph

neural architecture to model dependencies among API calls. Liu et al. [36] further built a sensitivity-aware heterogeneous graph and improved robustness through adversarial graph augmentation. These methods preserve structural information more effectively than pure sequence models, especially for process interactions. Their main limitation is that fine-grained parameter semantics are either weakly used or not explicitly modeled.

Another line of work aims to bridge the semantic gap between low-level API parameters and malicious behavior by introducing security sensitivity. Chen et al. [24] proposed parameter-based API tagging and sensitivity-inspired API embeddings, Liu et al. [37] evaluated the sensitivity of API event parameters to identify suspicious events, and Chen et al. [23] leveraged IOCs extracted from cyber threat intelligence to assign different security sensitivity levels to API calls. The advantage of these methods is improved semantic interpretability and better identification of security-relevant behaviors. However, they often rely on hand-crafted rules, external indicators, or manually designed sensitivity criteria, which may limit robustness and generalizability across datasets and environments.

Pretrained language models have been introduced to API-based malware analysis. Ding et al. [27] used BERT to encode process-level API sequences and then constructed directed process graphs for detection, while Trizna et al. [49] demonstrated the effectiveness of self-supervised Transformer models for malware detection and classification. These methods capture richer contextual dependencies than sparse or manually engineered representations. Nevertheless, APIs are still largely treated as tokens, and the relational semantics hidden in parameter values are not explicitly modeled.

Overall, existing studies have improved malware detection by modeling sequential context, execution structure, or parameter sensitivity. However, few methods jointly exploit fine-grained parameter semantics and explicit relational structure. In particular, the relational information carried by virtual address values in API parameters remains underexplored. Existing work either underuses parameter-induced relations or does not explicitly encode them as heterogeneous intent edges. Our work addresses this gap by constructing API relations from virtual address parameters and learning them with HGT.

3 Intent Subsequences Method

3.1 Principle of Intent Subsequences

The compiler translates the high-level intentions of the programmer into low-level code according to specific syntactic rules. In contrast, the software analysis process can be viewed as the reverse operation of compilation, often starting with the opcode or API, analyzing the characteristics of this data to infer program intent. Pygmalion [48] was one of the earliest successful systems that inferred recursive programs by analyzing a set of specific program executions.

As shown in Figure 1①, the intent to perform file writing represents a consistent pattern, consisting of three fixed steps (intents): I_1 to open a specific file, I_2

to write content, and I_3 to close the file. Depending on the programmer’s system platform, the intent is transformed into platform code ($B_1 - B_3$). As shown in Figure 1②, these steps are repeated to complete the entire code.

As shown in Figure 1③, when this software is executed in the sandbox environment, the raw API call sequence that can be captured is $A_1 - C_2$, and may include other functional segments such as $A_1 - A_3$ and $C_1 - C_2$. Further analysis of the `NtOpenFile` within the sequence reveals that the first parameter `filehandle` is a pre-allocated handle. This handle value alone holds no meaningful semantic information, as it merely represents a 32-bit (or 64-bit) virtual address value. Moreover, this virtual address value changes dynamically.

However, when jointly analyzing the API call sequence from B_1 to B_3 , it is observed that each API contains a parameter of type `filehandle`, and all these parameters have the same virtual address value. Despite this value dynamically changing across different execution periods, the matched values in the illustrated resource-use workflow refer to the same file handle. This characteristic allows for parameter matching based on both type and value, thereby extracting all APIs containing `filehandle` parameters with the same virtual address value, thus obtaining an intent subsequence $B_1 - B_3$ that captures the corresponding resource-use workflow. This method can also be applied to the Linux platform, requiring the substitution of the parameter `filehandle` with `fd`. As shown in Figure 1④, even if API calls D_n and E_n are inserted into the sequence, or if the sequence of API calls reported in the sandbox is not entirely accurate, we are still able to extract the intent subsequence $B_1 - B_3$. This characteristic is also evident in various library functions. For example, the minimum API call sequence required to set up OAuth authorization for a Twitter client is fixed. All other usage methods add extra API calls for optional functionality (e.g., SSL encryption) but all contain the minimal API call sequence as a subsequence [28].

3.2 Application to the Windows APIs

Given the focus of this paper on malware detection within the Windows platform, we apply the proposed principle of extracting intent subsequences to the Windows platform APIs. The process of extracting intent subsequences involves two steps:

Parameter Selection. Comprehensively analyzing Windows APIs is challenging due to their large number, the heterogeneity of parameter types, and the complexity of their semantics. Therefore, we focused our analysis on 707 commonly used Windows APIs that are hooked by the DRAKVUF sandbox [1]. Among the 406 NT-type system call APIs, we extracted parameters with identical types and names and tallied the frequency of these parameters across all APIs. Then, we retained parameter names whose values refer to the same logical resource across multiple APIs. For example, `HANDLE: FileHandle` represents operations on the same file, whereas `PVOID: BaseAddress` refers to operations on the same virtual address. Finally, 20 kernel-mode and 13 user-mode API parameter names were selected as key names. The detailed list is provided in Table 7 in the Appendix.

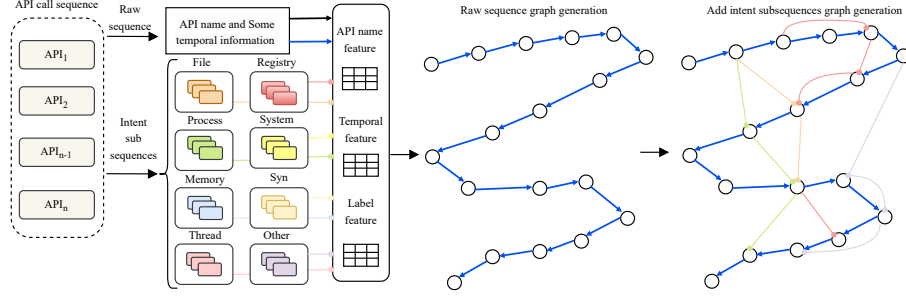


Fig. 2: Construction of the intent-aware heterogeneous graph. Nodes denote API call occurrences. Black edges represent raw temporal relations, and colored edges represent intent relations induced by matched key parameters.

The key-parameter set $\mathcal{K}$ is currently manually curated according to the API schemas exposed by the analyzed Windows environments and the resource semantics of individual parameters. Therefore, $\mathcal{K}$ should be regarded as environment- and API-coverage-dependent rather than universal.

Matching Rule. By applying the extraction rules defined in Algorithm 1, we group API calls that operate on the same logical resource. Specifically, for each API call, we first extract key parameters whose names belong to the selected key-parameter set, and normalize their values. API calls sharing the same normalized key-parameter name and value are assigned to the same ordered intent group $S_{k,v}$. For APIs whose return values semantically denote resource handles, the normalized return value is also used as a resource identifier. Finally, intent edges are generated from adjacent or fully connected members within each ordered group.

The matching procedure relies on observed resource identifiers and does not explicitly reconstruct the complete lifetime of each operating-system resource. Numerical handles are meaningful only within the corresponding execution context, so process boundaries should be considered when interpreting matched relations. These factors motivate us to treat the extracted subsequences as resource-level behavioral associations rather than ground-truth semantic annotations.

In terms of computational complexity, let N denote the number of API calls and $\bar{p}$ the average number of selected key parameters associated with each call. With hash-based indexing, construction of the resource groups requires approximately $O(N\bar{p})$ operations. When edges are generated only between adjacent members of each group, edge construction is linear in the number of matched occurrences. For fully connected groups, the worst-case number of generated edges is $O(\sum_{S_{k,v} \in \mathcal{S}} |S_{k,v}|^2)$. The subsequent graph-learning cost therefore depends on the resulting number of nodes and edges.

Algorithm 1 Matching algorithm for intent subsequences

Input: API call sequence X
Input: Set of parameter names of interest $\mathcal{K}$
Input: Set of APIs whose return values denote resource handles $\mathcal{R}$

- 1: $\mathcal{G} \leftarrow$ empty map from (k, v) to ordered lists
- 2: **for** $i = 0$ **to** $|X| - 1$ **do**
- 3: $A \leftarrow X[i]$
- 4: $P_A \leftarrow \{k : \text{normalize}(v) \mid (k, v) \in A \wedge k \in \mathcal{K}\}$
- 5: **for all** $k \in \text{keys}(P_A)$ **do**
- 6: $\mathcal{G}[(k, P_A[k])] \leftarrow \mathcal{G}[(k, P_A[k])] \parallel [A]$
- 7: **end for**
- 8: **if** $A \in \mathcal{R}$ **then**
- 9: $r_A \leftarrow \text{normalize}(\text{return}(A))$
- 10: $\mathcal{G}[(\text{return}, r_A)] \leftarrow \mathcal{G}[(\text{return}, r_A)] \parallel [A]$
- 11: **end if**
- 12: **end for**
- 13: $\mathcal{S} \leftarrow \{S_{k,v} \mid S_{k,v} = \mathcal{G}[(k, v)] \wedge |S_{k,v}| > 1\}$
- 14: $\mathcal{E}_{\text{intent}} \leftarrow$ edges generated from adjacent or fully connected members in each $S_{k,v} \in \mathcal{S}$

Output: Ordered intent groups $\mathcal{S}$ and intent edges $\mathcal{E}_{\text{intent}}$

4 Intent-Aware Malware Detection Architecture

4.1 Intent-Reliability Feature Engineering

API Name. Since the order of APIs in the call sequence does not appear randomly, it certainly embodies some contextual patterns [15]. Word2Vec [40], which captures contextual relations, has been widely used to learn API embeddings [15, 52]. In this paper, we employ the skip-gram model of Word2Vec, treating each API call as a word, and training it on a corpus to generate fixed-size feature vectors for each distinct word.

Intent Edge Type. We use naming conventions to infer coarse resource categories for intent edges⁴, this enriches the semantic information of the extracted intent subsequences. The type of the intent subsequence can be determined by the object or resource type indicated by the first API name. For example, `NtCreateFile` indicates that the object or resource is a `File`, and this intent subsequence is categorized under the file class. As shown in Figure 2, according to this classification method, we classify the extracted intent subsequences into eight categories: files, processes, memory, threads, registry, system, synchronization, and others. Along with the raw sequence, we obtain a total of nine categories. These data will be provided to the model for learning as features.

4.2 Intent-Heterogeneous Graph Construction

Given the strong information representation capability of graph structures, a heterogeneous directed graph can be employed to construct the API call se-

⁴ <https://learn.microsoft.com/en-us/windows/win32/api/>

quence. This graph can be represented as a triplet $G = (V, E, L)$, where V is a finite set of nodes, each node $v \in V$ represents an API call occurrence in the sequence. $E \subseteq V \times V$ denotes the set of directed edges, where an edge from node v_1 to v_2 indicates some form of contextual association between them. L is a multiset of labels in the graph, and $E \rightarrow L$ is a labeling function, which assigns a specific label to each edge based on the category represented by that edge. Consequently, we represent the API call sequence as a heterogeneous directed graph in a structured and fine-grained manner, which can further be analyzed using graph learning models.

4.3 Graph Learning

To ensure the reliability and robustness of malware detection against environment-induced noise, we employ a HGT architecture. This framework replaces traditional homogeneous aggregation with an intent-aware mechanism, specifically optimized for the nine edge categories (eight intents plus one raw sequence) defined in Section 4.2.

The model represents different resource-associated relations by learning type-specific projection matrices. For a source node u and target node v connected by an intent edge of type $e \in L$, the attention score is computed as:

$$\text{Att}(u, e, v) = \text{Softmax} \left(\frac{Q(v) \cdot W_e \cdot K(u)^T}{\sqrt{d}} \right) \quad (1)$$

where W_e is the weight matrix unique to each intent category (e.g., File, Registry, or Memory). This allows the model to prioritize reliable functional patterns (such as consistent file handle operations) while de-emphasizing volatile or redundant API calls.

Following the attention layers, a graph pooling (gPool) layer is utilized to distill high-level abstract features. The final graph representation is fed into a Multi-Layer Perceptron (MLP) with a Softmax output to generate the classification probability:

$$\hat{y} = \text{Softmax}(\text{MLP}(\text{gPool}(X, A))) \quad (2)$$

5 Evaluation

5.1 Datasets

Although several publicly available malware datasets exist [2–4, 16, 22, 46], they do not fully align with our requirements. For example, datasets [2, 16] only contain static features, datasets [4, 22, 46] provide only API call sequences without API parameters, and the malware samples in dataset [3] are not executable.

After selection, we chose two publicly available datasets, **Datacon** [5] and **Avast-CTU** [6], which include API parameters. Additionally, we constructed a

custom dataset **SeqMB**, which features a larger number of hook APIs and more comprehensive parameter analysis.

Datacon Dataset. This dataset contains 30,000 samples, including 20,000 benign and 10,000 malware. It did not disclose the malware families. The reports are provided in XML format, without recording adjacent or duplicate API calls and lacking inter-process relationships. This dataset includes 94 unique APIs.

Avast-CTU Dataset. This dataset is based on the CAPEv2 sandbox [7] and the reports are formatted in JSON. Each sample report includes not only API call sequences but also static information, file accesses, and network events. Due to the lack of benign reports in the original dataset, we collected benign samples from Chocolatey [8]. To verify that these Portable Executable (PE) files were indeed benign, we scanned all of them with VirusTotal [9], and removed any files that contained malware alerts. These benign samples were then executed in the same sandbox environment, resulting in a total of 39,322 benign reports. Table 1 presents the details of this dataset.

SeqMB Dataset. We constructed the SeqMB dataset, including malware and benign samples, following a two-phase process: PE file collection and API call sequence collection.

In the PE file collection phase, we obtained benign PE executable files from popular free software websites such as *portableapps.com*, *sourceforge.net*, and *softonic.com*. These files were also scanned using VirusTotal, and any files flagged for malware were removed. We obtained malware samples from VirusShare [10].

After collecting the PE files, it is necessary to use a sandbox to generate analysis reports. We found that most papers construct their reports using the Cuckoo sandbox [11]. However, after comparing different sandbox technologies, we discovered that the Cuckoo sandbox employs in-band monitoring agent technology, since this technology requires the execution of an agent process inside a virtual machine, it can easily be detected and terminated by malware. Considering that many malware authors have already employed various methods to detect the current analysis systems [33, 42, 51], aiming to evade analysis, we chose the DRAKVUF sandbox [1], a black-box binary analysis system that utilizes hardware-assisted virtualization to monitor virtual machines from outside, without requiring any additional agent within the virtual machine, it hooks API calls at the hypervisor layer. This analysis system hooks 406 NT-type system call APIs and 301 user-mode APIs, totaling 707 unique APIs. Compared to Cuckoo’s 332 unique APIs [12], this significantly enriches the features of the samples. Moreover, the two sandbox products also differ in their ability to record API parameters. Specifically, the Cuckoo sandbox [11] does not record the `OpenOptions` and `IoStatusBlock` parameters of the `NtOpenFile` API, whereas the DRAKVUF sandbox [1] is capable of capturing all parameters of this API.

According to Kuechler et al. [34], two minutes of execution is sufficient for malware to expose its malicious behavior. Therefore, we set the execution time of the samples in the sandbox to five minutes. The final dataset contains 25,123 benign and 30,837 malware, with the distribution of malware families shown in Table 1.

Table 1: Malware family distribution across SeqMB and Avast-CTU datasets.

Family	Type	SeqMB	Avast-CTU
Benign	Benign	25,123	39,322
Adload	Malware	–	704
Emotet	Malware	–	14,429
HarHar	Malware	–	655
Lokibot	Malware	–	4,191
njRAT	Malware	–	3,372
Qakbot	Malware	–	4,895
Swisyn	Malware	–	12,591
Trickbot	Malware	–	4,202
Ursnif	Malware	–	1,343
Zeus	Malware	–	2,594
AIadvise	Malware	704	–
Virus	Malware	4,429	–
Agent	Malware	655	–
Trojan	Malware	10,191	–
Ransom	Malware	3,372	–
Worm	Malware	4,895	–
Generic	Malware	6,591	–

Table 2: Statistics of report file sizes and the predefined truncation thresholds, where K denotes kilobytes.

Dataset	Type	Min	Max	Avg	Threshold
Datacon	Benign	0.6K	4,224K	172K	380K
	Malware	1.6K	3,883K	1,242K	
SeqMB	Benign	1.5K	251,673K	1,261K	1,400K
	Malware	3.1K	1,572,864K	59,528K	
Avast	Benign	18K	1,131,122K	7,261K	9,000K
	Malware	130K	8,888,930K	11,897K	

5.2 Data Preprocessing

Due to the varying formats and information across different datasets, some pre-processing is required before further feature extraction. This primarily involves four aspects: 1) Extracting a sample’s API calls and those from its subprocesses in the sandbox report. 2) Removing adjacent and duplicate API calls in the sequence. 3) Generating intent subsequences. 4) Unless otherwise specified, all datasets were stratified into training, validation, and testing subsets with proportions of 70%, 10%, and 20%, respectively.

As shown in Table 2, there is a significant difference in the file sizes of reports (lengths of API call sequences) between malware and benign. To prevent the model from over-relying on the simplistic feature of API call lengths, which could lead to “shortcut” phenomena [18], we empirically determined a truncation threshold for each dataset.

5.3 Environments and Hyperparameters

The DRAKVUF and CAPEv2 sandboxes were deployed on a server running Ubuntu 22.04 (64-bit), equipped with an Intel(R) Xeon(R) Gold 6230 CPU @ 2.10 GHz, 256 GB of RAM, and a 1 TB hard disk drive. Model training was conducted on an internal laboratory shared cloud platform running Ubuntu 22.04 (64-bit), equipped with an Intel(R) Xeon(R) Gold 6326 CPU @ 2.90 GHz, 64 GB of RAM, and two NVIDIA A100 80 GB GPUs. Some hyperparameters need to be set in our proposed model. Based on previous works [20, 35], we constrained the hyperparameters with the most significant effects on detection performance within a predefined search space, and then compared various combinations of these hyperparameters, through this approach, we identified an optimal hyperparameter configuration, as detailed in Table 3.

Table 3: Key hyperparameters and their optimal values for the model.

Model	Hyperparameter	Optimal Value
Word2Vec	Feature vector size	32
	Window size	12
HGT	Input layer and output layer	(128, 64)
	Number of attention heads	4
gPool	Top-k score	0.7
MLP	Units of hidden layers	(128, 64)
	Dropout rate	0.2

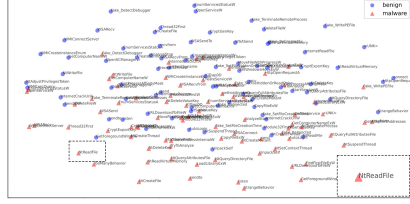
5.4 Metrics

Intrinsic Evaluation Metrics. The intent subsequence is the core of the proposed method. Therefore, cosine similarity is employed as the quantitative evaluation metric in our study, and the API calls’ vector similarity is defined as follows:

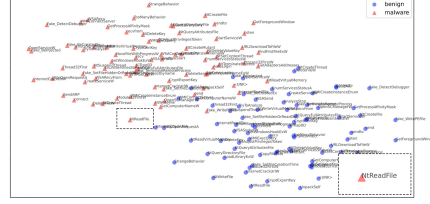
$$\text{similarity}(\mathbf{x}_a, \mathbf{x}_b) = \frac{\mathbf{x}_a \cdot \mathbf{x}_b}{\|\mathbf{x}_a\| \|\mathbf{x}_b\|} = \mathbf{x}_a \cdot \mathbf{x}_b \quad (3)$$

Then, the cosine similarity can be derived, as maximizing the dot product is equivalent to maximizing cosine similarity. This means that the cosine similarity can be used to measure similarity of API calls, with closer vectors indicating higher similarity. We use t-SNE for qualitative visualization.

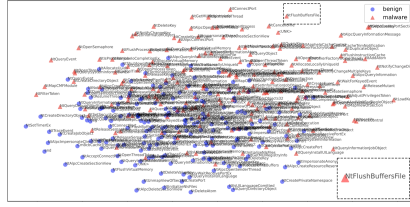
Extrinsic Evaluation Metrics. We evaluate model performance using four standard metrics: accuracy (ACC), precision (PR), recall (RC), and F1 score (F1), defined as $ACC = \frac{TP+TN}{TP+TN+FP+FN}$, $PR = \frac{TP}{TP+FP}$, $RC = \frac{TP}{TP+FN}$, and $F1 = \frac{2 \cdot PR \cdot RC}{PR+RC}$. Here, TP and TN represent correctly predicted malware and benign, respectively, while FP and FN denote benign predicted as malware and malware predicted as benign.



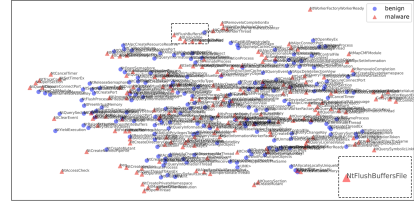
(a) Embeddings of raw API calls in the Datacon dataset.



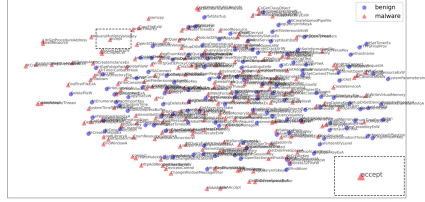
(b) Embeddings of intent API calls in the Datacon dataset.



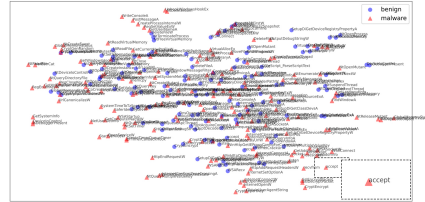
(c) Embeddings of raw API calls in the SeqMB dataset.



(d) Embeddings of intent API calls in the SeqMB dataset.



(e) Embeddings of raw API calls in the Avast dataset.



(f) Embeddings of intent API calls in the Avast dataset.

Fig. 3: Application of t-SNE to compare embeddings of raw and intent API call sequences.

Table 4: Comparison with baselines. All metrics are reported in %.

Method	Data	ACC	PR	RC	F1
Chen et al. [24]	Datacon	98.50	98.42	97.29	97.85
	SeqMB	88.50	86.02	88.33	87.16
	Avast	79.82	77.33	76.16	76.74
Zhang et al. [53]	Datacon	97.46	98.29	98.64	98.46
	SeqMB	79.30	79.00	76.76	77.86
	Avast	86.83	87.11	86.88	86.99
Ding et al. [27]	Datacon	94.28	95.08	96.31	95.69
	SeqMB	85.28	86.61	85.32	85.96
	Avast	81.98	83.08	80.37	81.70
Cui et al. [25]	Datacon	97.71	97.90	98.22	98.06
	SeqMB	90.21	91.80	89.22	90.49
	Avast	87.01	86.90	87.63	87.26
Proposed	Datacon	99.26	98.98	99.64	99.31
	SeqMB	92.03	91.45	92.15	91.80
	Avast	94.07	96.92	94.41	95.65

5.5 Evaluation Results

Intrinsic Evaluation We employed the Word2Vec method to embed API calls derived from both the raw sequences and those processed through the intent subsequence method. Using t-SNE techniques [38], we reduced the dimensionality of these embeddings to intuitively understand the clustering of embedded words in the lower-dimensional space.

As shown in Figure 3, the embeddings processed by the intent subsequence method exhibit more pronounced clustering compared with the raw sequences. This observation indicates that the intent subsequences enhance the correlation between API calls. Moreover, the SeqMB dataset was generated using the DRAKVUF sandbox, which features the largest number of API hooks, resulting in a significantly higher number of points compared to other datasets.

Notably, as shown in Figure 3a, the raw API token embeddings of malware and benign in the Datacon dataset exhibit substantial overlap. In contrast, Figure 3b shows that after processing with the intent subsequence method, the two classes become more separable in the low-dimensional visualization. This phenomenon indicates that the embeddings obtained through intent subsequence processing alone can effectively distinguish between malicious and benign behaviors. The absence of such a clear separation in the SeqMB and Avast datasets is attributed to their broader behavioral feature coverage, which requires higher-dimensional feature modeling for effective classification.

This representation-level evaluation measures the usefulness of the extracted resource associations for feature learning rather than the semantic correctness of each individual subsequence against independently annotated ground truth.

Table 5: Ablation study on the contribution of intent edges.

Model variant	Datacon F1 (%)	SeqMB F1 (%)	Avast F1 (%)
Node-only HGT	95.46	85.97	75.18
HGT + raw temporal edges only	95.32	84.22	73.21
HGT + all edges	99.31	91.80	95.65

Extrinsic Evaluation Most studies rely on self-constructed datasets, of which only 3.8% are publicly accessible [29]. This results in a lack of standardized dataset benchmarks, hindering the comparison of results across different papers. To assess the baseline performance of detection frameworks, we prioritized datasets that are highly relevant to our research task and are publicly available. Specifically, Ding et al. [27] and Chen et al. [24] utilized the same open-source datasets as this study, Cui et al. [25] made their code publicly available. Additionally, Zhang et al. [53], Ding et al. [27], and Chen et al. [24] provided their code upon our request.

To ensure a fair comparison, we adopted the open-source hyperparameter optimization framework Optuna [14] to conduct a unified search over the key hyperparameters of all baseline models, such as the learning rate and hidden layer dimension. The best configurations were then selected based on validation set performance for final evaluation. For the other datasets, we likewise report the best results achieved by each baseline method under its corresponding optimal settings.

As shown in Table 4, the proposed method achieves the best performance on all three datasets, namely Datacon, SeqMB, and Avast, demonstrating its consistent performance across three datasets. Specifically, on the Datacon dataset, our method attains the highest classification performance, with ACC and F1 scores reaching 99.26% and 99.31%, respectively. In contrast, the SeqMB and Avast datasets contain broader and more complex sample behavioral characteristics, with greater variation in behavioral patterns, leading to performance drops of varying degrees for most methods on these datasets. Nevertheless, our method still achieves F1 scores of 91.80% and 95.65%, respectively, indicating that the proposed approach exhibits good within-dataset generalization under the current splits.

We note that evaluation on multiple independent datasets should not be interpreted as direct cross-dataset generalization, because each model is trained and tested within the corresponding dataset.

Ablation Study To evaluate the contribution of the proposed intent subsequence mechanism, we conduct an ablation study by comparing three model variants: the basic HGT model, HGT with only raw temporal edges, and HGT with both raw temporal and intent edges (HGT + all edges). The basic HGT mainly learns malware-related patterns from API call representations, while the raw temporal-edge variant additionally incorporates the original execution order

Table 6: Temporal concept-drift comparison on Avast-CTU: F1 scores (%).

Method	Jul.–Aug.	Sep.–Oct.	Nov.–Dec.
	2019	2019	2019
Chen et al. [24]	90.92	89.37	86.15
Zhang et al. [53]	94.31	93.10	90.54
Ding et al. [27]	92.76	91.44	88.61
Cui et al. [25]	94.88	94.14	91.62
Proposed	97.42	97.54	95.83

of API calls. The intent-edge variant further introduces the semantic relations extracted from matched virtual-address-related parameters.

As shown in Table 5, adding raw temporal edges does not bring consistent improvement over the Node-only HGT model. Specifically, the F1 scores of HGT are 95.46%, 85.97%, and 75.18% on Datacon, SeqMB, and Avast, respectively, while the corresponding scores of HGT with raw temporal edges are 95.32%, 84.22%, and 73.21%. This indicates that raw temporal relations alone provide limited additional benefit. A possible reason is that the basic HGT can already capture discriminative API usage patterns from node representations, whereas raw temporal edges mainly reflect the observed execution order. Such order information may be noisy, interleaved, or unstable in sandbox-generated traces, and therefore cannot reliably represent high-level malicious behavior.

In contrast, introducing intent edges leads to substantial performance gains across all datasets. Compared with HGT using only raw temporal edges, HGT with all edges improves the F1 score from 95.32% to 99.31% on Datacon, from 84.22% to 91.80% on SeqMB, and from 73.21% to 95.65% on Avast. These results demonstrate that intent edges provide much richer semantic information than raw temporal edges. By connecting API calls that operate on the same logical resource, such as files, processes, memory regions, or registry objects, the proposed intent subsequences explicitly encode resource-level behavioral dependencies. These dependencies are less dependent on the observed execution order and provide complementary resource-level behavioral information for malware detection.

Robustness When the distribution of input data in the testing phase is inconsistent with the training data, the assumption of independent and identically distributed (i.i.d.) data no longer holds, leading to concept drift. This causes temporal and spatial biases in model evaluation, leading to an overestimation of the malware classifier’s performance [39, 41, 43, 44].

To evaluate the expected robustness of the model in real-world environments, as shown in Figure 4, we referred to the research of Pendlebury et al. [43] and introduced three constraints to the Avast dataset: temporal training consistency, temporal malware and benign window consistency, and a realistic malware-to-

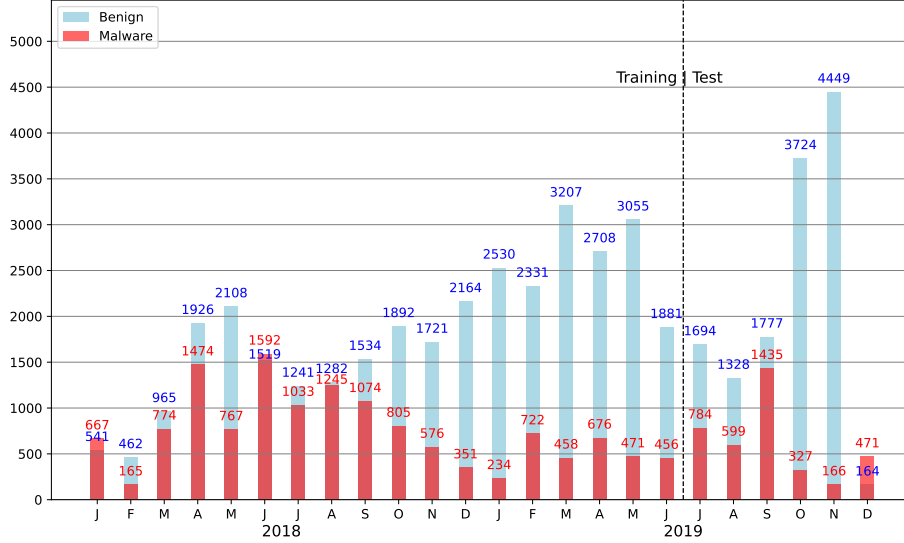


Fig. 4: The figure shows the bar chart of the monthly distribution of the Avast dataset, spanning January 2018 to December 2019. The dashed vertical line indicates the temporal boundary between the training and test periods.

benign ratio in testing. The dataset was restructured, and evaluations of temporal biases were conducted separately.

By repartitioning the dataset, we ensured that all samples in the training set strictly preceded those in the test set, and came from the same time window, thus maintaining real-world temporal consistency in the evaluation. To assess the impact of temporal bias, we divided the test set into three subsets, each covering different periods from July to December 2019, at two-month intervals. We extended this temporal robustness evaluation to include the four baseline methods used in the main comparison. All methods are trained using the same temporally preceding training data and evaluated on the same three future test windows, with identical data partitions and evaluation protocols.

As shown in Table 6, the evaluated methods exhibit varying degrees of performance change under temporal distribution shift, confirming that concept drift is a common challenge for API-based malware detection. The proposed method maintains F1 scores of 97.42%, 97.54%, and 95.83% on the three future test windows, respectively. These results indicate that the proposed resource-associated relations remain useful under temporal drift, while the decreasing performance at larger temporal distances also shows that the method does not eliminate distribution shift.

The results also show an overall tendency of performance degradation as the temporal distance from the training period increases. This observation is consistent with the presence of temporal distribution shift and evolving malware behaviors.

6 Limitations and Future Work

One of the central assumptions in machine learning is that the training and test data are drawn identically and independently from the same underlying data distribution [47]. However, in the cybersecurity domain, discrepancies often arise between training and deployment data distributions. Over time, the deployment data distribution gradually diverges from the training data, leading to degradation in classifier performance. Bias remains a persistent challenge for machine learning classifiers in cybersecurity [18, 21]. While it is nearly impossible to completely eliminate such bias, we acknowledge that our experiments are also subject to its influence.

In the future, we will further investigate the impact of biases on different models. Current designs are predominantly supervised and heavily rely on high-quality training datasets to construct accurate and robust detection models. However, collecting high-quality training data is a challenging task. For instance, the labels assigned to malware by VirusTotal may vary across reports published in different years. We believe it is feasible to mitigate the effects of bias without altering the dataset itself.

The current implementation relies on a manually curated key-parameter set tied to the API schemas of the analyzed Windows environments. If a different sandbox exposes substantially different APIs or parameter schemas, this set may need to be remapped or extended.

Another limitation concerns ambiguity in resource identifiers. Handle values may be reused during execution, and process-related information is not equally available across all datasets. The current method does not explicitly reconstruct complete resource lifetimes; therefore, a small number of extracted associations may reflect identifier reuse rather than continuous access to the same logical resource.

Although the proposed method is evaluated independently on three datasets, the current experiments do not constitute direct train-on-one/test-on-another cross-dataset evaluation. The datasets differ in sandbox instrumentation, hooked API coverage, parameter availability, and report format. Consequently, direct transfer would additionally require alignment of the observable API and parameter spaces. Cross-sandbox and cross-dataset adaptation will be investigated in future work.

7 Conclusion

In this paper, we propose the intent subsequence method for processing API call parameters, using otherwise unstable virtual-address values to construct resource-associated contextual relations among API calls. Our evaluation shows that these additional resource-associated relations improve representation learning and downstream malware classification. These resource-associated relations provide important features for API-call-based malware analysis tasks, thereby improving classification performance. We believe that this method can also benefit other software analysis tasks based on API call sequences.

Acknowledgments. This work was supported by the Youth Innovation Promotion Association CAS (2023 Member Project, Grant No. E3YY031104).

Disclosure of Interests. The authors have no competing interests to declare that are relevant to the content of this article.

Appendix

Table 7: API parameters selected as key names.

Type	Category	Name	Description
System	File	FileHandle	Identifies a specific file.
		LinkHandle	Identifies a specific link.
	Process	ProcessHandle	Identifies a specific process.
		TokenHandle	Identifies a specific Access Token.
		PortHandle	Identifies a specific port.
	Memory	BaseAddress	Starting point of the memory region.
		SectionHandle	Maps a specified memory region.
	Thread	ThreadHandle	Identifies a specific thread.
	Registry	KeyHandle	Identifies a specific registry.
		TargetKey	Identifies a specific TargetKey.
	System	TimerHandle	Identifies a specific timer.
		VariableName	System environment variables.
		ObjectAttributes	Describes the created object.
		ClientToken	Identifies a specific Security Token.
	Syn	Event	An event object.
		EventHandle	Identifies a specific event.
		MutantHandle	Identifies a specific mutant.
	Other	hWnd	Identifies a specific window.
		Handle	References a kernel object.
		HandleId	Identifies resources or objects.
User	Registry	hkey	Handle to the registry key.
		phkey	Handle to a key.
	Other	handle	References an object.
		hinternet	Specific Internet connection.
		socket	Identifies a network session.
		lpsockaddr	Points to SOCKADDR structure.
		sc_handle	Service control handle.
		pvoid	Pointer to data of any type.
		lpvoid	Long pointer to data of any type.
		hcryptkey	Identifies a specific key entity.
		lpwstr	Pointer to a wide character.
		lpcwstr	Pointer to a const wide character.
		lpctstr	Pointer to a TCHAR character.

References

1. <https://drakvuf.com/>
2. <https://www.kaggle.com/datasets/blackarcher/malware-dataset>, accessed: 2024-09-22
3. <https://www.kaggle.com/competitions/malware-classification/data>, accessed: 2024-09-22
4. <https://zenodo.org/record/1203289>, accessed: 2024-09-22
5. <https://github.com/kericwy1337/Datacon2019-Malicious-Code-DataSet-Stage1>, accessed: 2024-10-22
6. <https://github.com/avast/avast-ctu-cape-dataset>, accessed: 2024-10-25
7. <https://github.com/kevoreilly/CAPEv2>, accessed: 2024-10-22
8. <https://chocolatey.org>, accessed: 2024-11-10
9. <https://www.virustotal.com/gui/home/upload>, accessed: 2024-11-22
10. <https://virusshare.com/>, accessed: 2024-09-22
11. <https://cuckoosandbox.org/>
12. <https://github.com/cuckoosandbox/cuckoo/wiki/Hooked-APIs-and-Categories>, accessed: 2024-12-22
13. Agrawal, R., Stokes, J.W., Marinescu, M., Selvaraj, K.: Neural sequential malware detection with parameters. In: 2018 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP). pp. 2656–2660. IEEE (2018)
14. Akiba, T., Sano, S., Yanase, T., Ohta, T., Koyama, M.: Optuna: A next-generation hyperparameter optimization framework. In: Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining (2019)
15. Amer, E., Zelinka, I.: A dynamic Windows malware detection and prediction method based on contextual understanding of API call sequence. *Computers & Security* **92**, 101760 (2020). <https://doi.org/10.1016/j.cose.2020.101760>
16. Anderson, H.S., Roth, P.: Ember: an open dataset for training static pe malware machine learning models. *arXiv preprint arXiv:1804.04637* (2018)
17. Aonzo, S., Han, Y., Mantovani, A., Balzarotti, D.: Humans vs. machines in malware classification. In: 32nd USENIX Security Symposium (USENIX Security 23). pp. 1145–1162 (2023)
18. Arp, D., Quiring, E., Pendlebury, F., Warnecke, A., Pierazzi, F., Wressnegger, C., Cavallaro, L., Rieck, K.: Dos and don'ts of machine learning in computer security. In: 31st USENIX Security Symposium (USENIX Security 22). pp. 3971–3988. USENIX Association (Aug 2022)
19. Aslan, Ö.A., Samet, R.: A comprehensive review on malware detection approaches. *IEEE Access* **8**, 6249–6271 (2020). <https://doi.org/10.1109/ACCESS.2019.2963724>
20. Bilot, T., El Madhoun, N., Al Agha, K., Zouaoui, A.: A survey on malware detection with graph representation learning. *ACM Comput. Surv.* (may 2024)
21. Botacin, M., Ceschin, F., Sun, R., Oliveira, D., Grégio, A.: Challenges and pitfalls in malware research. *Computers & Security* **106**, 102287 (2021)
22. Catak, F.O., Yazı, A.F.: A benchmark api call dataset for windows pe malware classification. *arXiv preprint arXiv:1905.01999* (2019)
23. Chen, T., Zeng, H., Lv, M., Zhu, T.: Ctimd: Cyber threat intelligence enhanced malware detection using api call sequences with parameters. *Computers & Security* **136**, 103518 (2024)

24. Chen, X., Hao, Z., Li, L., Cui, L., Zhu, Y., Ding, Z., Liu, Y.: Cruparamer: Learning on parameter-augmented api sequences for malware detection. *IEEE Transactions on Information Forensics and Security* **17**, 788–803 (2022)
25. Cui, L., Cui, J., Ji, Y., Hao, Z., Li, L., Ding, Z.: Api2vec: Learning representations of api sequences for malware detection. In: *Proceedings of the 32nd ACM SIGSOFT International Symposium on Software Testing and Analysis*. p. 261–273. ISSTA 2023, Association for Computing Machinery (2023). <https://doi.org/10.1145/3597926.3598054>
26. Dambra, S., Han, Y., Aonzo, S., Kotzias, P., Vitale, A., Caballero, J., Balzarotti, D., Bilge, L.: Decoding the secrets of machine learning in malware classification: A deep dive into datasets, feature extraction, and model performance. In: *Proceedings of the 2023 ACM SIGSAC Conference on Computer and Communications Security*. pp. 60–74 (2023)
27. Ding, Z., Xu, H., Guo, Y., Yan, L., Cui, L., Hao, Z.: Mal-bert-gcn: Malware detection by combining bert and gcn. In: *2022 IEEE International Conference on Trust, Security and Privacy in Computing and Communications (TrustCom)*. pp. 175–183 (2022). <https://doi.org/10.1109/TrustCom56396.2022.00034>
28. Fowkes, J., Sutton, C.: Parameter-free probabilistic api mining across github. In: *Proceedings of the 2016 24th ACM SIGSOFT international symposium on foundations of software engineering*. pp. 254–265 (2016)
29. Grajeda, C., Breiteringer, F., Baggili, I.: Availability of datasets for digital forensics—and what is missing. *Digital Investigation* **22**, S94–S105 (2017)
30. Hu, Z., Dong, Y., Wang, K., Sun, Y.: Heterogeneous graph transformer. In: *Proceedings of The Web Conference 2020*. pp. 2704–2710. Association for Computing Machinery (2020). <https://doi.org/10.1145/3366423.3380027>
31. Jindal, C., Salls, C., Aghakhani, H., Long, K., Kruegel, C., Vigna, G.: Neurflux: dynamic malware analysis without feature engineering. In: *Proceedings of the 35th Annual Computer Security Applications Conference*. p. 444–455. ACSAC ’19 (2019). <https://doi.org/10.1145/3359789.3359835>
32. Kirat, D., Vigna, G.: Malgene: Automatic extraction of malware analysis evasion signature. In: *Proceedings of the 22nd ACM SIGSAC Conference on Computer and Communications Security*. p. 769–780. CCS ’15, Association for Computing Machinery (2015). <https://doi.org/10.1145/2810103.2813642>
33. Kirat, D., Vigna, G., Kruegel, C.: BareCloud: Bare-metal analysis-based evasive malware detection. In: *23rd USENIX Security Symposium (USENIX Security 14)*. pp. 287–301. USENIX Association, San Diego, CA (Aug 2014)
34. Kuechler, A., Mantovani, A., Han, Y., Bilge, L., Balzarotti, D.: Does every second count? time-based evolution of malware behavior in sandboxes. In: *Network and Distributed System Security Symposium* (2021), <https://api.semanticscholar.org/CorpusID:231798972>
35. Li, C., Cheng, Z., Zhu, H., Wang, L., Lv, Q., Wang, Y., Li, N., Sun, D.: DMalNet: Dynamic malware analysis based on API feature engineering and graph learning. *Computers & Security* **122**, 102872 (2022). <https://doi.org/10.1016/j.cose.2022.102872>
36. Liu, C., Li, B., Zhao, J., Feng, W., Liu, X., Li, C.: A2-clm: Few-shot malware detection based on adversarial heterogeneous graph augmentation. *IEEE Transactions on Information Forensics and Security* (2023)
37. Liu, C., Li, B., Zhao, J., Liu, X., Li, C.: Malaf: Malware attack foretelling from run-time behavior graph sequence. *IEEE Transactions on Dependable and Secure Computing* (2023)

38. Van der Maaten, L., Hinton, G.: Visualizing data using t-sne. *Journal of machine learning research* **9**(11) (2008)
39. Maniriho, P., Mahmood, A.N., Chowdhury, M.J.M.: Api-maldetect: Automated malware detection framework for windows based on api calls and deep learning techniques. *Journal of Network and Computer Applications* **218**, 103704 (2023)
40. Mikolov, T., Chen, K., Corrado, G., Dean, J.: Efficient estimation of word representations in vector space. *International Conference on Learning Representations* (Jan 2013)
41. Mimura, M.: Impact of benign sample size on binary classification accuracy. *Expert Systems with Applications* **211**, 118630 (2023)
42. Pék, G., Bencsáth, B., Buttyán, L.: nether: In-guest detection of out-of-the-guest malware analyzers. In: *Proceedings of the Fourth European Workshop on System Security*. pp. 1–6 (2011)
43. Pendlebury, F., Pierazzi, F., Jordaney, R., Kinder, J., Cavallaro, L.: {TESSERACT}: Eliminating experimental bias in malware classification across space and time. In: *28th USENIX security symposium (USENIX Security 19)*. pp. 729–746 (2019)
44. Rossow, C., Dietrich, C.J., Grier, C., Kreibich, C., Paxson, V., Pohlmann, N., Bos, H., Van Steen, M.: Prudent practices for designing malware experiments: Status quo and outlook. In: *2012 IEEE symposium on security and privacy*. pp. 65–79. IEEE (2012)
45. Salehi, Z., Sami, A., Ghiasi, M.: Maar: Robust features to detect malicious activity based on api calls, their arguments and return values. *Engineering Applications of Artificial Intelligence* **59**, 93–102 (2017)
46. Severi, G., Leek, T., Dolan-Gavitt, B.: Malrec: compact full-trace malware recording for retrospective deep analysis. In: *Detection of Intrusions and Malware, and Vulnerability Assessment: 15th International Conference, DIMVA 2018, Saclay, France, June 28–29, 2018, Proceedings 15*. pp. 3–23. Springer (2018)
47. Shalev-Shwartz, S., Ben-David, S.: *Understanding machine learning: From theory to algorithms*. Cambridge university press (2014)
48. Smith, D.C.: *Pygmalion: a creative programming environment*. Stanford University (1975)
49. Trizna, D., Demetrio, L., Biggio, B., Roli, F.: Nebula: Self-attention for dynamic malware analysis. *IEEE Transactions on Information Forensics and Security* **19**, 6155–6167 (2024). <https://doi.org/10.1109/TIFS.2024.3409083>
50. Xie, Y., Luo, X., Sun, J.: Towards enhancing sequence-optimized malware representation with context-separated bi-directional long short-term memory and proximal policy optimization. *IEEE Transactions on Dependable and Secure Computing* pp. 1–12 (jan 2024). <https://doi.org/10.1109/TDSC.2024.3352604>
51. Yin, H., Lou, B., Reiher, P.: A method for summarizing and classifying evasive malware. In: *Proceedings of the 26th International Symposium on Research in Attacks, Intrusions and Defenses*. pp. 455–470 (2023)
52. Zhang, S., Wu, J., Zhang, M., Yang, W.: Dynamic malware analysis based on api sequence semantic fusion. *Applied Sciences* **13**(11), 6526 (2023)
53. Zhang, Z., Qi, P., Wang, W.: Dynamic malware analysis with feature engineering and feature learning. In: *Proceedings of the AAAI conference on artificial intelligence*. vol. 34, pp. 1210–1217 (2020)